KUBERNETES PRACTICAL EXPERIMENT

Analyze Resource Allocation, Scheduling, and Load Balancing in a Three-Node Cluster

1. Aim

To analyze resource allocation, pod scheduling, and load balancing in an existing three-node Kubernetes cluster by deploying applications, configuring CPU and memory requests and limits, observing pod distribution, and testing Service load balancing.

2. Requirements

- An existing three-node Kubernetes cluster.
- kubectl configured and connected to the cluster.
- Internet connectivity to download container images.
- PowerShell or a terminal.

3. Step-by-Step Procedure

Step 1: Verify the Kubernetes cluster

```
kubectl get nodes -o wide
```

- Displays all nodes available in the cluster.
- Verify that all three nodes are in the Ready state.
- Note down the node names.

Step 2: Analyze available resources

```
kubectl top nodes
```

- Displays CPU and memory utilization of each node.
- Compare resource utilization across all three nodes.
- If metrics are unavailable, check whether Metrics Server is installed using the command below.

```
kubectl get deployment metrics-server -n kube-system
```

Step 3: Create a deployment

Create a file named **deployment.yaml** and add the following configuration:

```
apiVersion: apps/v1
kind: Deployment
metadata:
name: resource-demo
spec:
replicas: 6
selector:
matchLabels:
app: resource-demo
template:
metadata:
labels:
app: resource-demo
spec:
containers:
- name: nginx
image: nginx
resources:
requests:
cpu: "100m"
memory: "64Mi"
limits:
cpu: "200m"
memory: "128Mi"
```

```
kubectl apply -f deployment.yaml
```

- Creates a Deployment with six replicas, each running an Nginx container.
- A CPU request of 100m represents 0.1 CPU core; the memory request is 64 MiB.
- CPU and memory limits define the maximum resources the container may consume.
- The scheduler uses resource requests when deciding where to place pods.

Step 4: Analyze pod scheduling

```
kubectl get pods -o wide
kubectl get events --sort-by=.metadata.creationTimestamp
```

- The NODE column identifies the node hosting each pod.
- Observe how the six pods are distributed across the three nodes.
- Scheduling depends on available resources, node conditions, and scheduling constraints.
- Events can provide information about successful scheduling or scheduling failures.

Step 5: Analyze resource allocation

```
kubectl describe nodes
kubectl top pods
```

- Node details show CPU and memory capacity and allocated resources.
- Review the allocated-resource section to understand requested resources on each node.
- The top command displays current pod CPU and memory consumption.

Step 6: Test pod distribution with more replicas

```
kubectl scale deployment resource-demo --replicas=12
kubectl get pods -o wide
```

- Increases the number of replicas from six to twelve.
- Kubernetes schedules the additional pods according to available resources and scheduling constraints.
- Compare the new pod distribution with the six-replica deployment.

Step 7: Create a Kubernetes Service for load balancing

Create a file named **service.yaml** and add the following configuration:

```
apiVersion: v1
kind: Service
metadata:
name: nginx-service
spec:
type: NodePort
selector:
app: resource-demo
ports:
- port: 80
targetPort: 80
nodePort: 30080
```

```
kubectl apply -f service.yaml
```

- Creates a NodePort Service that selects pods with the label app: resource-demo.
- Port 30080 is exposed on cluster nodes, subject to cluster networking and firewall rules.
- The Service forwards connections to eligible backend pod endpoints.
- Kubernetes Service load balancing distributes connections across available endpoints; it does not guarantee equal request counts per pod.

Step 8: Verify load-balancing endpoints

```
kubectl get services
kubectl get endpoints nginx-service
kubectl get endpointslices
```

- Displays the Service and its assigned port.
- Endpoints and EndpointSlices show the pod IP addresses registered as backends.

- Check that the expected healthy application pods are registered.

Step 9: Test the application

Open a browser and enter the following address, replacing NODE-IP with an IP address reachable from your computer:

```
http://NODE-IP:30080
```

- Accesses the Nginx application through the NodePort Service.
- The Service forwards the connection to an available backend pod.
- If permitted by the cluster network, test using the IP addresses of different nodes.
- A successful page response confirms application access through the Service, but does not by itself prove equal traffic distribution.

Step 10: Observe resource utilization again

```
kubectl top nodes
kubectl top pods
```

- Observe CPU and memory utilization after increasing the replicas.
- Compare resource consumption before and after scaling.
- Identify which nodes host the workloads and compare their utilization.
- Remember that pod placement and actual CPU or memory usage are different measurements.

Step 11: Clean up resources

```
kubectl delete service nginx-service
kubectl delete deployment resource-demo
```

- Deletes the Service and Deployment created for this experiment.
- The associated pods are removed.
- The existing three-node cluster remains intact.

4. Observation Table

Experiment	Observation to record
Cluster verification	Status of all three nodes
Initial resource utilization	CPU and memory usage of each node
Six replicas	Number of pods scheduled on each node
Twelve replicas	Updated pod distribution across nodes
Resource allocation	CPU and memory requests and limits
Load balancing	Service backend endpoints and application response
Final utilization	CPU and memory usage after scaling

5. Expected Learning Outcomes

- Understand how Kubernetes allocates CPU and memory resources.
- Understand resource requests and limits.
- Observe how the Kubernetes scheduler assigns pods to nodes.
- Analyze how scaling affects pod distribution.
- Understand how Kubernetes Services distribute connections across backend pods.
- Monitor node and pod resource utilization.

6. Result

Resource allocation, pod scheduling, and Service-based load balancing were analyzed using the existing three-node Kubernetes cluster.

7. Suggested Practical Extension

Increase the CPU request in the Deployment and observe what happens when the cluster does not have enough allocatable CPU to schedule a pod. Use **kubectl get pods -o wide** and **kubectl describe pod POD-NAME** to inspect a Pending pod and its scheduling events. This demonstrates resource-based scheduling constraints.